

Unit-II: Server-Side Programming with Python (8 hours)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

Introduction to functions

- In Python, a function is a reusable block of code designed to perform a specific task.
- Functions help organize code, making it more readable, maintainable, and efficient by avoiding repetition.

```
def say_hello():  
    print("Hello from a function")
```

Types of Functions

- Built-in functions: Python provides several built-in functions like `print()`, `len()`, `max()`, etc.
- User-defined functions: Functions created by the user to suit their specific needs.
- Lambda functions: Anonymous, small, single-expression functions defined using the `lambda` keyword.

Defining a Function

- A function is defined using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`
- The code block within the function is indented.

```
def say_hello():  
    print("Hello, Programmer!")
```

Calling a Function

- To execute a function, you call it by its name followed by parentheses.

`say_hello()` # Output: Hello Programmer!

Function Arguments

- Functions can take arguments (inputs) to operate on.
- These are specified within the parentheses in the function definition.

```
def greet(name):  
    print(f"Hello, {name} welcome!")
```

```
greet("Sunil") # Output: Hello, Sunil Welcome!  
greet("NAST") # Output: Hello, NAST Welcome!  
greet("BCA") # Output: Hello, BCA Welcome!
```

Return Statement

- The return statement is used to exit a function and return a value.
- If no return statement is present, the function returns None.

#Defining

```
def add(x, y):  
    """This function adds two numbers."""  
    return x + y
```

#Calling

```
result = add(5, 3)  
print(result) # Output: 8
```

Scope and global variables

- In programming, scope determines where a variable is accessible within a program.
- Global variables have a global scope, meaning they can be accessed from anywhere in the program, while local variables have a limited scope, typically within a function or block of code.

Scope and global variables

Scope:

- Scope defines the visibility and accessibility of a variable.
- It determines which parts of the code can access and modify a variable.

Global Variables:

- Global variables are declared outside of any function or block of code.
- They are accessible from anywhere within the program.

Local Variables:

- Local variables are declared within a function or a block of code.
- Their scope is limited to that function or block, meaning they cannot be accessed from outside it.

Scope and global variables

```
global_var = 10
```

```
def my_function():  
    local_var = 5  
    print(local_var) # Output: 5  
    print(global_var) # Output: 10
```

```
my_function()  
# print(local_var) # This would cause an error, as local_var is not defined  
# outside the function  
print(global_var) # Output: 10
```

Scope and global variables

Shadowing:

- If a local variable has the same name as a global variable, the local variable will "shadow" the global variable within the scope of the function or block, meaning the local variable takes precedence.

Python Lambda Function

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

Syntax:

lambda arguments : expression

- The expression is executed and the result is returned:

Python Lambda Function

Example

- Add 10 to argument a, and return the result:

```
x = lambda a : a + 10
```

```
print(x(5))
```

Python Lambda Function

- Lambda functions can take any number of arguments for **Example**,
- Multiply argument a with argument b and return the result:

```
x = lambda a, b : a * b  
print(x(5, 6))
```

Example

- Summarize argument a, b, and c and return the result:

```
x = lambda a, b, c : a + b + c  
print(x(5, 6, 2))
```